

SQLCipher for Android Migration (android-database-sqlcipher to sqlcipher-android)

The original SQLCipher for Android library (i.e., `android-database-sqlcipher`) was introduced in 2011 (<https://www.zetetic.net/blog/2011/11/30/sqlcipher-for-android-released.html>). While we have continued to maintain and improve the original library over the years, there were features we and others wanted that were not easy to integrate within the existing library.

We introduced a long-term replacement API (i.e., `sqlcipher-android` (</sqlcipher/sqlcipher-for-android/>)) in 2022 (<https://www.zetetic.net/blog/2022/05/11/sqlcipher-android-refresh/>). This document aims to help highlight the specific changes necessary to transition to from the older `android-database-sqlcipher` to the new `sqlcipher-android` (</sqlcipher/sqlcipher-for-android/>) library.

Commercial edition licenses includes full database encryption and:

- Optimized performance (up to 4x faster than Community)
- Pre-built packages for quickest integration
- Value Level Encryption functions (</sqlcipher/value-level-encryption/>)
- Encrypted Virtual Tables (</sqlcipher/encrypted-virtual-tables/>)
- Performance Counters & Statistics (</sqlcipher/sqlcipher-stats/>)
- Benchmark & Performance Testing Tools (</sqlcipher/sqlcipher-perf/>)
- Private, priority support

The Commercial edition of SQLCipher for Android can be purchased directly from the Zetetic Store. Licensed software is delivered immediately upon payment.

[Buy SQLCipher for Android Now » \(/sqlcipher/commercial/\)](/sqlcipher/commercial/)

Below are the 3 main areas that will require adjustments within existing codebases to begin utilizing the new API.

Gradle Configuration

Community Edition

Change the old AAR reference from:

```
implementation 'net.zetetic:android-database-sqlcipher:4.5.4@aar'  
implementation 'androidx.sqlite:sqlite:2.2.0'
```

To the new AAR reference:

```
implementation(group: 'net.zetetic', name: 'sqlcipher-android', version: '4.11.0')  
implementation(group: 'androidx.sqlite', name: 'sqlite', version: '2.2.0')
```

Commercial Edition

Change the old AAR reference from:

```
implementation (name: 'android-database-sqlcipher-4.5.4', ext: 'aar')  
implementation 'androidx.sqlite:sqlite:2.2.0'
```

To the new AAR reference:

```
implementation fileTree(dir: 'libs', include: ['sqlcipher-android-4.11.0.aar'])  
implementation(group: 'androidx.sqlite', name: 'sqlite', version: '2.2.0')
```

Package Adjustments

The `android-database-sqlcipher` library stored the majority of classes within the `net.sqlcipher` and `net.sqlcipher.database` packages. Classes generally parallel between the two libraries, however, `sqlcipher-android` uses `net.zetetic.database` and `net.zetetic.database.sqlcipher`.

API Adjustments

Native Library Loading

The native SQLCipher library must be loaded into memory prior to any database operation. The `android-database-sqlcipher` library used a static function to do this:

```
SQLiteDatabase.loadLibs(this);
```

Using `sqlcipher-android`, the library loading is simplified, which makes it easier to use alternative mechanisms for native library loading (e.g., `SplitInstallHelper.loadLibrary` ([https://developer.android.com/reference/com/google/android/play/core/splitinstall/SplitInstallHelper.html#loadLibrary\(android.content.Context,%20java.lang.String\)\)](https://developer.android.com/reference/com/google/android/play/core/splitinstall/SplitInstallHelper.html#loadLibrary(android.content.Context,%20java.lang.String)))).

```
System.loadLibrary("sqlcipher");
```

Constructor Differences

In `android-database-sqlcipher`, when using the static `openOrCreateDatabase` call to acquire a `SQLiteDatabase` instance, this version of the method accepted a `DatabaseErrorHandler` as the fifth argument:

```
SQLiteDatabase database = SQLiteDatabase.openOrCreateDatabase(databaseFile.getAbsolutePath(), "test  
key", null, hook, null);
```

In `sqlcipher-android`, the `DatabaseErrorHandler` parameter moves to the fourth parameter:

```
SQLiteDatabase database = SQLiteDatabase.openOrCreateDatabase(databaseFile.getAbsolutePath(), "test  
key", null, null, hook);
```

Pre & Post Key Operations

Coordinating operations that occur before and after the keying operation have changed slightly between the two API's. Both libraries utilize an `SQLiteDatabaseHook` as a mechanism to inject operations into the pre and post-key operations. The type provided to the function for both `preKey` and `postKey` changed. `android-database-sqlcipher` would provide the instance of the `SQLiteDatabase` object, whereas `sqlcipher-android` provides the underlying `SQLiteConnection` object.

Room API

Both `android-database-sqlcipher` and `sqlcipher-android` support the Room persistence library API. In both cases, SQLCipher would integrate within the `Room.databaseBuilder` (<https://developer.android.com/reference/androidx/room/>

`Room#databaseBuilder(kotlin.String,kotlin.Function0)`), specifically using the

`OpenHelperFactory(...)` (<https://developer.android.com/reference/androidx/room/>

`RoomDatabase.Builder#OpenHelperFactory(androidx.sqlite.db.SupportSQLiteOpenHelper.Factory)`).

The `android-database-sqlcipher` library would provide an instance of the `SupportFactory` (<https://github.com/sqlcipher/android-database-sqlcipher/blob/master/android-database-sqlcipher/src/main/java/net/sqlcipher/database/SupportFactory.java>), whereas

`sqlcipher-android` provides the `SupportOpenHelperFactory` (<https://github.com/sqlcipher/sqlcipher-android/blob/master/sqlcipher/src/main/java/net/zetetic/database/sqlcipher/SupportOpenHelperFactory.java>).

© 2025, Zetetic, LLC (<https://www.zetetic.net>). All rights reserved. | [Security \(/security/\)](#) | [Privacy \(/privacy/\)](#) | [Contact \(/contact/\)](#)

Follow @TeamZetetic (<https://twitter.com/TeamZetetic>)

Zetetic Newsletter (<https://confirmsubscription.com/h/y/2EC1451D94FE09EF>)